

InfraLynx Middleware & Viva Preparation Notes

■InfraLynx Middleware & Viva Preparation Notes

1. Why full functionality is required before middleware

The faculty instruction clearly states that the application must be fully functional before middleware is implemented. This means that middleware is only an additional layer on top of a working application. If the core product is incomplete, then middleware cannot compensate for missing business features. A complete web application is expected to include the main features of the system, such as user dashboards, admin module, revenue model, and core workflows. Middleware like logging, error handling, upload handling, and security are evaluation aspects, but they are not a substitute for the actual working application.

In simple words: the app should work first, and then middleware should make it secure, monitored, and robust.

2. Middleware explanation

Middleware is code that sits between a client request and the application logic. It intercepts requests before the controller/route logic is executed. Middleware is used for tasks such as logging, validation, security checks, rate limiting, request transformation, and file upload handling.

In NestJS, middleware is typically used to perform common logic for all requests. It helps centralize concerns that should apply across multiple routes. In this project, custom middleware is used to log incoming requests and limit excessive API usage.

The app also uses exception handling and file logging to manage errors and keep records of what happened in the application.

3. Helmet and CORS explanation

Helmet is a security middleware package for Express/NestJS applications. It adds security headers to every HTTP response so the browser uses safer behavior. It helps protect against common vulnerabilities like clickjacking and unsafe resource loading. In this project, Helmet is added in `main.ts` with the following idea: it secures the HTTP response by adding headers such as CSP-related protection and related security parameters.

CORS stands for Cross-Origin Resource Sharing. It allows the browser to request resources from another domain/origin when that server permits it. In this project, the backend and frontend are separated, so CORS is required for frontend pages to communicate with the backend. It is enabled in `main.ts` with origin: `'*'`, which allows requests from any origin in the prototype environment. In production, origin should usually be restricted to trusted domains.

4. File-by-file breakdown

`main.ts`

- This is the application bootstrap file.
- It creates the NestJS app.
- It applies Helmet security middleware.
- It enables CORS.
- It serves uploaded files from the uploads folder.
- It applies global validation pipes.
- It sets up Swagger documentation for the API.
- It starts the app on port 3000.

`app.module.ts`

- This file wires the global middleware.
- It applies `RateLimitMiddleware` and `LoggerMiddleware` for all routes.

- It also registers the global role guard and exception filter.

logger.middleware.ts

- Logs HTTP method, route, request role, response status, and response time.
- Records request details for monitoring and debugging.

rate-limit.middleware.ts

- Tracks requests by client IP.
- Limits requests in a fixed time window.
- If the limit is exceeded, it returns 429 Too Many Requests.

all-exceptions.filter.ts

- Catches all exceptions.
- Converts them into structured JSON responses.
- Logs error details so they can be inspected later.

file-logger.service.ts

- Buffers logs in memory.
- Writes logs to files at regular intervals.
- Keeps access logs and error logs in separate files under back-end/logs.

uploads.controller.ts / uploads module

- Allows file upload functionality.
- Handles receiving and storing uploaded files.
- File upload handling is also an important part of the evaluation criteria.

5. Viva answer format

Answer 1: What is middleware?

Middleware is code that runs between the client request and the controller. It helps with logging, validation, security, rate limiting, and file handling before the request reaches the business logic.

Answer 2: Why is middleware important?

Middleware centralizes cross-cutting concerns such as security and logging, making the application more consistent, easier to maintain, and more secure.

Answer 3: What is Helmet?

Helmet adds security headers to HTTP responses. It helps reduce common web vulnerabilities by improving browser security behavior.

Answer 4: What is CORS?

CORS allows a frontend running on one origin to call an API running on another origin. In this project, it is necessary because the frontend and backend are separate.

Answer 5: Why is full functionality required before middleware?

Because middleware should be implemented on top of a working application. The evaluation expects the complete app to be functional, with core features and modules working correctly.

Answer 6: What files are relevant?

The important files are app.module.ts, main.ts, logger.middleware.ts, rate-limit.middleware.ts, all-exceptions.filter.ts, file-logger.service.ts, and the uploads module/controller.

Final viva summary

Our project uses middleware to handle monitoring, security, and request control. Helmet protects the app by adding HTTP security headers, while CORS enables frontend-backend communication. The app also includes route-level middleware, file logging, and exception handling to align with evaluation criteria. However, the main idea is that the project must already be fully functional before layering middleware on top of it.

Evaluation mindset

The faculty expects: complete app + proper middleware + logging + file-based error management + upload and security handling. So the project should not be treated as middleware-only; it should be seen as a complete web application with middleware added as a required quality layer.